

Complete End-to-End Pipeline Execution Guide

Make sure you always source your environment first in every new terminal window:

Shell

```
source ~/env.sh
```

Phase 1: Level 1 – Basic Flight, SITL & Python Waypoints

Goal: Verify that ArduPilot SITL, MAVLink, and Python automation scripts can successfully arm and control the drone.

Terminal 1: Start ArduPilot SITL

Launch the direct ArduPilot binary using the JSON backend model:

Shell

```
source ~/env.sh
cd ~/ardupilot
./build/sitl/bin/arducopter --model gazebo-iris --speedup 1
--sim-address=127.0.0.1 -I0
```

Terminal 2: MAVProxy Telemetry & Port Routing

Connect MAVProxy and configure the secondary telemetry port required for external scripts:

Shell

```
source ~/env.sh
mavproxy.py --master=tcp:127.0.0.1:5760 --out 172.28.48.1:14550
--map --console
```

At the MAVProxy prompt (**MAV>**), type:

Shell

```
output add 127.0.0.1:14560
```

Terminal 3: Run Automated Flight Script

Execute the Python pymavlink script to automatically switch to **GUIDED** mode, arm motors, take off, and fly through predefined local NED coordinates:

Shell

```
source ~/env.sh  
python3 level1_fly_points.py
```

Phase 2: Level 2 – 3D Maze Simulation, LiDAR Bridge, & Cartographer SLAM

Goal: Spawn the drone inside the 3D maze, bridge LiDAR data to ROS 2, generate a live 2D occupancy grid map via Cartographer, and save it.

Terminal 1: Launch Gazebo Maze World (01_gazebo.sh)

Start the Gazebo simulation server and GUI with the OGRE rendering engine to prevent WSL crashes:

Shell

```
./01_gazebo.sh
```

(Wait here until the Gazebo window loads and the Iris quadcopter is fully visible in the maze.)

Terminal 2: Boot ArduPilot SITL (02_sitl.sh)

Start the autopilot backend:

Shell

```
./02_sitl.sh
```

At the MAVProxy prompt (**STABILIZE>**), type to link telemetry for ROS:

Shell

```
output add 127.0.0.1:14560
```

Terminal 3: Start ROS 2 Sensor Bridge (03_bridge.sh)

Bridge Gazebo sensor topics (/lidar -> /scan and odometry) into ROS 2 format:

Shell

```
./03_bridge.sh
```

Terminal 4: Launch Cartographer SLAM & RViz (04_cartographer.sh)

Publish static TF transforms (base_link to imu_link / base_scan) and fire up Cartographer with RViz:

Shell

```
./04_cartographer.sh
```

(In RViz, ensure the Fixed Frame is set to map.)

Terminal 5: Fly the Drone & Save the Map (05_save_map.sh)

1. Command the drone to fly through the maze corridors (using MAVProxy or your flight script).
2. Once mapping is complete, run the save script to generate map.png:

Shell

```
./05_save_map.sh
```

Phase 3: Level 3 – Autonomous Navigation (Nav2 Integration)

Goal: Use the generated map to let the drone navigate autonomously via the ROS 2 Navigation2 stack.

Terminals 1 to 3: Keep Simulation & Bridges Running

Ensure Gazebo (01_gazebo.sh), SITL (02_sitl.sh), and the ROS 2 Bridge (03_bridge.sh) are actively running in their respective terminals.

Terminal 4: Launch ROS 2 Nav2 Stack

Instead of Cartographer, boot up the Navigation2 bringup node using simulation time:

Shell

```
source ~/env.sh  
ros2 launch nav2_bringup navigation_launch.py use_sim_time:=true
```